BỘ GIÁO DỤC VÀ ĐÀO TẠO

HỌC PHẦN ĐỒ ÁN CAPSTONE (ECO2415) — BỘ MÔN THƯƠNG MẠI ĐIỆN TỬ

BÁO CÁO TỔNG KẾT ĐỒ ÁN CAPSTONE

ĐỀ TÀI 13: HỆ THỐNG QUẢN LÝ KHO BÁN LẺ & BÁN HÀNG TẠI QUẦY (POS)

KIẾN TRÚC CLEAN ARCHITECTURE 4 TẦNG & MICROSOFT SQL SERVER

Học phần:	Đồ án Capstone Thương Mại Điện Tử (ECO2415)
Mã đề tài:	Đề tài số 13 (Kho bán lẻ: Nhập – Xuất – Tồn)
Kiến trúc:	Clean Architecture 4 Tầng (.NET 8 Blazor Server)
Cơ sở dữ liệu:	Microsoft SQL Server (Chuẩn 3NF, Dapper & 1 Transaction)
Điểm tự đánh giá:	**100 / 100 Điểm (Đạt đủ 17 mã K1.1 – K5.3)**

HỌC KỲ CAPSTONE 2026

BÁO CÁO TỔNG KẾT ĐỒ ÁN CAPSTONE (ECO2415)

HỆ THỐNG QUẢN LÝ KHO BÁN LẺ & BÁN HÀNG TẠI QUẦY (POS)

WEBKHOBANLE — CLEAN ARCHITECTURE 4 TẦNG & SQL SERVER

Thông Tin Đồ Án:

- **Tên đề tài:** Hệ thống Quản lý Kho Bán Lẻ (Nhập – Xuất – Tồn) & Bán Hàng Tại Quầy (POS)
- **Mã đề tài:** Đề tài số 13 (Căn cứ văn bản *18 Đề tài Đồ án Capstone — ECO2415, Bộ môn Thương Mại Điện Tử*)
- **Mô hình kiến trúc:** Clean Architecture 4 Tầng (*CoreBusiness* → *UseCases* → *DataStore Plugin* → *WebApp*)

- **Nền tảng công nghệ:** .NET 8 LTS, C#, ASP.NET Core Blazor Interactive Server, Microsoft SQL Server (SSMS), Dapper Micro-ORM, Bootstrap 5 & Sapo Theme Modern UI
- **Phiên bản tài liệu:** 1.0 — Mã tệp: `baocaodemo.md`

MỤC LỤC TỔNG QUAN

1. CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI & PHẠM VI NGHIỆP VỤ
    - 1.1. Bối cảnh thực tế & Tính cấp thiết của bài toán Kho Bán Lẻ
    - 1.2. Mục tiêu nghiên cứu và phát triển phần mềm
    - 1.3. Bảng ánh xạ nghiệp vụ từ eShop sang Kho Bán Lẻ (Mục 7.13 & Mục 8)
    - 1.4. Đối tượng sử dụng và phân quyền hệ thống (RBAC)
2. CHƯƠNG 2: PHƯƠNG PHÁP LUẬN & QUY TRÌNH PHÁT TRIỂN DỰ ÁN
    - 2.1. Phương pháp phát triển Agile/Scrum qua 3 Sprint
    - 2.2. Lịch trình phân rã công việc & Lịch sử mã nguồn Git
    - 2.3. Quy trình bảo đảm chất lượng (QA/QC) và Kiểm thử
3. CHƯƠNG 3: KIẾN TRÚC HỆ THỐNG — CLEAN ARCHITECTURE 4 TẦNG
    - 3.1. Tổng quan kiến trúc Clean Architecture & Nguyên tắc Dependency Rule
    - 3.2. Cấu trúc chi tiết 4 tầng Project trong giải pháp Visual Studio
    - 3.3. Cơ chế hoán đổi Data Plugin (Swappable Plugin: SQL Server & In-Memory)
    - 3.4. Quản lý trạng thái và vòng đời Service (Scoped, Transient, Singleton)
4. CHƯƠNG 4: THIẾT KẾ CƠ SỞ DỮ LIỆU & QUAN HỆ THỰC THỂ (ERD)
    - 4.1. Chuẩn hóa dữ liệu 3NF & Cấu trúc 9 bảng dữ liệu
    - 4.2. Mối quan hệ khóa ngoại (10 Foreign Keys) & Toàn vẹn tham chiếu
    - 4.3. Computed Columns, Ràng buộc CHECK và Chỉ mục tối ưu (Indexes)
    - 4.4. Các Views báo cáo và Stored Procedure bán lẻ POS trong 1 Transaction
5. CHƯƠNG 5: CÀI ĐẶT 2 LUẬT NGHIỆP VỤ CỐT LÕI (CORE BUSINESS RULES)

    - 5.1. LUẬT 1: Duyệt mới đổi tồn & Chặn xuất âm kho (K1.3, K3.3)
    - 5.2. LUẬT 2: Bảo toàn công thức Nhập - Xuất - Tồn & Đối soát chéo sổ cái (K1.3, K4.3)
    - 5.3. Đảm bảo tính toàn vẹn giao dịch Header-Line qua 1 Database Transaction (K2.2)
6. CHƯƠNG 6: CHI TIẾT CÁC PHÂN HỆ CHỨC NĂNG CỦA HỆ THỐNG
    - 6.1. Phân hệ Bán hàng tại quầy (POS) & In Bill nhiệt K80
    - 6.2. Phân hệ Quản lý Nhập kho & Trợ lý AI SmartPaste (Mục 10)
    - 6.3. Phân hệ Quản lý Xuất kho & Kiểm soát phiếu
    - 6.4. Phân hệ Quản lý Danh mục hàng hóa & Cảnh báo tồn kho tối thiểu
    - 6.5. Phân hệ Báo cáo Thống kê NXT & Xuất dữ liệu thật ra Excel
    - 6.6. Phân hệ Bảo mật, Phân quyền & Phòng thủ URL (K4.1, K4.2)
7. CHƯƠNG 7: BẢNG ĐỐI CHIẾU 17 MÃ KIỂM TRA PHIẾU CHẤM (K1.1 – K5.3)
8. CHƯƠNG 8: HƯỚNG DẪN CÀI ĐẶT, VẬN HÀNH & KỊCH BẢN DEMO BẢO VỆ
    - 8.1. Hướng dẫn thiết lập môi trường và chạy dự án
    - 8.2. Kịch bản Demo 5 bước thuyết phục Hội đồng chấm thi
    - 8.3. Bộ câu hỏi & Đáp án chuẩn bị vấn đáp tốt nghiệp (Oral Defense Q&A)
9. CHƯƠNG 9: KẾT LUẬN & HƯỚNG PHÁT TRIỂN

CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI & PHẠM VI NGHIỆP VỤ

1.1. Bối cảnh thực tế & Tính cấp thiết của bài toán Kho Bán Lẻ

Trong hoạt động kinh doanh bán lẻ (siêu thị mini, chuỗi cửa hàng tiện lợi, tiệm tạp hóa hiện đại), công tác quản lý kho hàng đóng vai trò sống còn. Một nghịch lý thường xảy ra tại các cửa hàng bán lẻ truyền thống:

- Hàng hóa trên phần mềm báo "còn hàng" nhưng khi khách đến quầy tìm lại hết, hoặc ngược lại phần mềm báo hết nhưng hàng vẫn nằm trong kho.
- Hiện tượng **xuất âm kho**: Do nhân viên bán buôn hoặc xuất điều chuyển tạo phiếu mà không kiểm tra lượng tồn thực tế, dẫn đến số tồn bị âm vô lý ($CurrentStock < 0$), phá vỡ toàn bộ sổ sách tài chính kế toán.
- **Mất cân đối giữa kho tổng và quầy thu ngân (POS)**: Thu ngân bán lẻ thu tiền của khách nhưng giao dịch không đồng bộ ngay lập tức vào thẻ kho, tạo ra độ trễ (latency) gây thất thoát hoặc bán trùng sản phẩm cuối cùng.

- **Nhập liệu thủ công chậm trễ, sai sót**: Khi nhà cung cấp giao hàng kèm theo hóa đơn/báo giá dạng văn bản, nhân viên kho mất hàng giờ gõ từng dòng mã hàng, số lượng, đơn giá, rất dễ nhầm lẫn số 0 hoặc nhầm đơn vị tính.

Xuất phát từ thực tiễn trên, dự án **WebKhoBanLe (Đề tài 13 — Học phần Đồ án Capstone ECO2415)** được nghiên cứu và hiện thực hóa nhằm cung cấp một giải pháp quản lý kho chuyên nghiệp, tích hợp liền mạch điểm bán hàng POS, bảo toàn số liệu tuyệt đối và tự động hóa quy trình nhập liệu thông qua trí tuệ nhân tạo.

1.2. Mục tiêu nghiên cứu và phát triển phần mềm

1. **Chuẩn hóa kiến trúc phần mềm**: Xây dựng ứng dụng web quản lý theo chuẩn **Clean Architecture 4 tầng** (do Robert C. Martin - Uncle Bob đề xuất), bảo đảm tính độc lập giữa nghiệp vụ cốt lõi và công nghệ hiển thị/lưu trữ.
2. **Triệt tiêu sai lệch kho**: Cài đặt cứng 2 luật nghiệp vụ bất biến: **Chặn xuất âm kho** và **Bảo toàn công thức Nhập - Xuất - Tồn**, bảo vệ cơ sở dữ liệu ở cấp độ cao nhất.
3. **Toàn vẹn dữ liệu trong môi trường cạnh tranh (Concurrency)**: Mọi thao tác xuất/nhập/thanh toán POS liên quan đến cặp chứng từ Header - Line đều được bảo vệ trong **1 Database Transaction** đạt chuẩn thuộc tính ACID.
4. **Trải nghiệm bán lẻ chuyên nghiệp**: Cung cấp giao diện POS mượt mà, hỗ trợ đa hình thức thanh toán (Tiền mặt tính tiền thừa tự động, quét mã VietQR tự động điền tiền, thẻ POS ngân hàng) và in phiếu hóa đơn nhiệt chuẩn K80.
5. **Tích hợp trí tuệ nhân tạo (AI SmartPaste)**: Hiện thực hóa tiêu chuẩn AI Mục 10 của học phần Capstone với cơ chế Human-in-the-loop, chi phí 0 đồng, bảo vệ thông tin và chuyển đổi báo giá phi cấu trúc thành dữ liệu có cấu trúc.

1.3. Bảng ánh xạ nghiệp vụ từ eShop sang Kho Bán Lẻ (Mục 7.13 & Mục 8)

Theo hướng dẫn của Bộ môn Thương Mại Điện Tử, mọi đề tài Capstone đều xuất phát từ nguyên lý *"eShop đổi miền nghiệp vụ, giữ nguyên khung 4 tầng Clean Architecture, thay đổi Entity, Luật nghiệp vụ, SQL và Giao diện"*. Bảng ánh xạ sau đây chứng minh sự chuyển đổi chuẩn mực của Đề tài 13:

Khái Niệm eShop Gốc (Video Bài Giảng)	Ánh Xạ Đề Tài 13: Kho Bán Lẻ (KhoBanLe)	Vai Trò & Nghiệp Vụ Cụ Thể Trong Dự Án
Product (Sản phẩm)	`Product` (Hàng hóa kho & quầy)	Quản lý SKU, Tên, ĐVT, Danh mục, Giá bán lẻ, Tồn kho tức thời (`CurrentStock`), Định mức tồn tối thiểu (`MinStockLevel`), Ảnh sản phẩm.

Khái Niệm eShop Gốc (Video Bài Giảng)	Ánh Xạ Đề Tài 13: Kho Bán Lẻ (KhoBanLe)	Vai Trò & Nghiệp Vụ Cụ Thể Trong Dự Án
Customer (Khách hàng)	`Customer` & `Supplier`	Quản lý Nhà cung cấp cho luồng Nhập kho và Khách hàng tích điểm cho luồng Bán lẻ POS.
Shopping Cart (Giỏ hàng)	`DraftReceiptState` & `ShoppingCartService`	Quản lý danh sách mặt hàng nháp trên trình duyệt (sử dụng dịch vụ `Scoped` theo kết nối Circuit của Blazor Server) trước khi lưu CSDL.
Order / LineItem (Đơn hàng)	`GoodsReceipt / Line` `GoodsIssue / Line` `PosOrder / Line`	Cặp Header – Line quản lý: Nhập kho từ NCC, Xuất kho nội bộ/bán sỉ, và Hóa đơn bán lẻ quầy thu ngân.
Admin Order Processing	**Phê duyệt phiếu & Đối soát** (`ApprovalList`, `InventoryReport`)	Quản lý kho duyệt phiếu chuyển trạng thái từ `Draft` → `Approved`, kích hoạt cập nhật tồn kho trong 1 Transaction an toàn.

1.4. Đối tượng sử dụng và phân quyền hệ thống (RBAC)

Hệ thống thiết kế theo cơ chế Phân quyền dựa trên vai trò (Role-Based Access Control - RBAC) với 2 nhóm người dùng chính:

1. **Thủ Kho (`WarehouseStaff`):**
    - Tra cứu danh mục hàng hóa, kiểm tra tồn kho.
    - Thao tác tại quầy bán lẻ POS: tạo đơn hàng, chọn thanh toán, in hóa đơn nhiệt.
    - Lập phiếu nhập kho (hỗ trợ nhập tay và sử dụng AI SmartPaste).
    - Lập phiếu xuất kho ở trạng thái Chờ duyệt (`Draft`).
    - Theo dõi trạng thái sổ phiếu nhập/xuất cá nhân.
    - *Quyền hạn bị giới hạn:* Không thể tự ý duyệt phiếu xuất để thay đổi tồn kho; không thể truy cập các trang quản trị nhạy cảm.
2. **Quản Lý Kho (`InventoryManager`):**
    - Toàn quyền xem và kiểm duyệt các phiếu nhập kho, phiếu xuất kho (`/approvals`).

- Thực thi thẩm định số lượng: hệ thống tự động kiểm tra tồn kho trước khi cho phép bấm nút duyệt.
- Xem báo cáo tổng hợp Nhập – Xuất – Tồn (`/reports`), kích hoạt công cụ đối soát chéo độc lập.
- Xuất dữ liệu báo cáo ra tệp bảng tính Excel/CSV UTF-8.
- Quản trị danh mục hàng hóa: Thêm mới, chỉnh sửa thông tin giá và định mức an toàn (`MinStockLevel`).

CHƯƠNG 2: PHƯƠNG PHÁP LUẬN & QUY TRÌNH PHÁT TRIỂN DỰ ÁN

2.1. Phương pháp phát triển Agile/Scrum qua 3 Sprint

Dự án được xây dựng theo mô hình phát triển phần mềm lặp và tăng trưởng (Iterative and Incremental Development) thông qua 3 Sprint với các cột mốc thời gian rõ ràng, đảm bảo tiêu chí đánh giá K5.3 (commit trải dài trên ít nhất 3 tuần):

```
gantt
    title TIẾN ĐỘ PHÁT TRIỂN DỰ ÁN KHO BÁN LẺ (3 SPRINTS)
    dateFormat  YYYY-MM-DD
    section Sprint 1: Cốt Lõi & Kiến Trúc
    Khởi tạo Clean Architecture 4 Tầng :done, s1_1, 2026-09-28, 4d
    Entity CoreBusiness & Repository Interfaces :done, s1_2, 2026-10-01, 3d
    DataStore In-Memory Mock Plugin :done, s1_3, 2026-10-03, 3d
    section Sprint 2: CSDL & Nghiệp Vụ Sâu
    Thiết kế 9 Bảng 3NF trên SQL Server :done, s2_1, 2026-10-06, 3d
    Cài đặt Dapper Micro-ORM & 1 Transaction :done, s2_2, 2026-10-08, 4d
    Hiện thực Luật 1 (Chặn xuất âm) & Luật 2 (Bảo toàn NXT) :done, s2_3, 2026-10-1
1, 4d
    section Sprint 3: POS, AI & Bảo Vệ
    Giao diện Sapo Modern & Bán Lẻ POS K80 :done, s3_1, 2026-10-15, 2d
    Tích hợp VietQR, In hóa đơn & Xuất Excel :done, s3_2, 2026-10-16, 2d
    AI SmartPaste Mục 10, Hồ sơ UML & GRADING.md :done, s3_3, 2026-10-19, 2d
```

Phân tích chi tiết từng Sprint:

- **Sprint 1 (Khởi tạo nền tảng & Dependency Rule)**:
  - Khởi tạo Solution với 4 Project độc lập. Thiết lập quy tắc tham chiếu nghiêm ngặt: `CoreBusiness` độc lập tuyệt đối; `UseCases` chỉ trỏ về `CoreBusiness`.
  - Tạo các Entity cốt lõi (`Product`, `GoodsReceipt`, `GoodsIssue`).
  - Viết bộ Plugin `DataStore.HardCoded` dưới dạng In-Memory Collection để phục vụ chạy thử nghiệm cục bộ mà chưa cần phụ thuộc vào hệ quản trị CSDL.
- **Sprint 2 (Hiện thực hóa Cơ sở dữ liệu & Cài đặt 2 Luật Nghiệp Vụ)**:
  - Thiết kế Schema SQL Server với 9 bảng chuẩn 3NF, bổ sung bảng `PosOrder`, `Customer`.
  - Viết bộ Plugin `KhoBanLe.DataStore.HardCoded/Sql/` sử dụng Dapper, thực thi 100% truy vấn tham số hóa (K2.3).
  - Cài đặt `ApproveGoodsIssueUseCase` kiểm tra chặn xuất âm kho và gói gọn cập nhật Header-Line trong khối `connection.BeginTransaction()` (K2.2).
  - Cài đặt `GetInventoryReportUseCase` và thuộc tính `EndingStock => BeginningStock + TotalImport - TotalExport` (Luật 2).
- **Sprint 3 (Bán lẻ POS, Trí tuệ nhân tạo, Tối ưu hóa UI/UX & Chuẩn bị bảo vệ)**:
  - Nâng cấp giao diện theo phong cách Sapo/Enterprise SaaS, thiết kế linh hoạt hỗ trợ màn hình di động 390px (K3.4).
  - Xây dựng phân hệ bán hàng tại quầy POS: tính tiền thừa, tạo mã VietQR động, in bill khổ K80.
  - Tích hợp Module AI SmartPaste trích xuất báo giá tự động tuân thủ 5 tiêu chuẩn Mục 10.
  - Xây dựng hồ sơ sơ đồ UML (Use Case, Class, Sequence) bằng PlantUML, hoàn thiện trang tương tác `/guide` và bộ tài liệu `GRADING.md`.

2.2. Lịch trình phân rã công việc & Lịch sử mã nguồn Git

Nhật ký commit của dự án phản ánh trung thực quá trình phát triển bền bỉ và chuẩn mực:

- `07694ee` (2026-09-28): *Sprint 1: Khoi tao du an Clean Architecture 4 tang, CoreBusiness va UseCases co ban*
- `939e266` (2026-10-06): *Sprint 2: Tich hop SQL Server Dapper, 2 Luat nghiep vu Nhap Xuat Ton va Transaction K2.2*
- `2aaa208` (2026-10-15): *Sprint 3: Nang cap toan dien giao dien phong cach Sapo Theme chuyen nghiep, ban hang POS & hoan tat bao ve do an*

- `a78ceba` (2026-10-15): *Refactor UI: Nang cap toan dien Dashboard & Header phong cach Enterprise SaaS khong loi layout*
- `4244ff8` (2026-10-16): *feat(pos): nâng cấp giao diện bán lẻ, in bill nhiệt K80, xuất Excel, đa phương thức thanh toán (Tiền mặt, VietQR, Thẻ POS), cập nhật 29 mặt hàng siêu thị và fix chuẩn font chữ/ảnh packshot*
- `1f84202` (2026-10-19): *feat: cap nhat hinh anh chuan va bo sung cac mat hang snack tap hoa Orion, Swing, O'Star, Toonies*

2.3. Quy trình bảo đảm chất lượng (QA/QC) và Kiểm thử

Hệ thống được kiểm tra nghiêm ngặt qua 3 cấp độ:

1. **Kiểm thử logic nghiệp vụ (Unit Testing / Use Case Level)**: Thử nghiệm độc lập logic của `ApproveGoodsIssueUseCase` với các ca kiểm thử biên (Số lượng xuất bằng tồn kho, xuất vượt tồn kho 1 đơn vị, xuất tồn kho bằng 0).
2. **Kiểm thử tích hợp dữ liệu (Integration / Database Level)**: Kiểm tra các tình huống Rollback giao dịch khi cố tình ngắt kết nối hoặc dữ liệu dòng hàng bị sai kiểu dữ liệu giữa chừng trong khối Transaction.
3. **Kiểm thử giao diện người dùng (End-to-End UI & Responsive Testing)**: Kiểm thử toàn bộ quy trình từ lúc thêm giỏ hàng POS → Thanh toán → Tồn kho giảm → Vào Báo cáo đối soát thấy số liệu khớp 100%. Kiểm thử trên công cụ DevTools ở độ phân giải di động iPhone 14/15 (390px).

CHƯƠNG 3: KIẾN TRÚC HỆ THỐNG — CLEAN ARCHITECTURE 4 TẦNG

3.1. Tổng quan kiến trúc Clean Architecture & Nguyên tắc Dependency Rule

Hệ thống WebKhoBanLe áp dụng triệt để kiến trúc Clean Architecture với quy tắc bất biến **Dependency Rule**: *Mọi sự phụ thuộc mã nguồn chỉ được phép hướng từ ngoài vào trong tâm. Các tầng bên trong không bao giờ được biết đến sự tồn tại của các tầng bên ngoài.*

```
graph TD

    UI["4. TẦNG WEB APP (KhoBanLe)<br/>• Blazor Server Components<br/>• Razor Page
s & Layouts<br/>• Composition Root (Program.cs)<br/>• State Services (Scoped)"]



    DATA["3. TẦNG DATASTORE / PLUGINS<br/>(KhoBanLe.DataStore.HardCoded)<br/>• Sql
ProductRepository (Dapper)<br/>• SqlGoodsReceiptRepository (Transaction)<br/>• Sql
GoodsIssueRepository (Transaction)<br/>• In-Memory Mock Plugins"]
```

```
    UC["2. TẦNG USE CASES (KhoBanLe.UseCases)<br/>• IProductRepository, IGoodsIssu
eRepository...<br/>• ApproveGoodsIssueUseCase (LUẬT 1)<br/>• GetInventoryReportUse
Case (LUẬT 2)<br/>• ParseSupplierQuoteUseCase (AI)"]


    CORE["1. TẦNG CORE BUSINESS (KhoBanLe.CoreBusiness)<br/>• Product, Supplier, C
ustomer<br/>• GoodsReceipt, GoodsReceiptLine<br/>• GoodsIssue, GoodsIssueLine<br/>
• PosOrder, PosOrderLine<br/>• InventoryReportItem, ReceiptStatus"]


    UI -->|Tham chiếu| UC

    UI -->|Composition Root| DATA

    UI -->|Tham chiếu| CORE

    DATA -->|Cài đặt Interface| UC

    DATA -->|Sử dụng Entity| CORE

    UC -->|Chỉ tham chiếu duy nhất| CORE


    style CORE fill:#f8fafc,stroke:#3b82f6,stroke-width:3px

    style UC fill:#f1f5f9,stroke:#8b5cf6,stroke-width:2px

    style DATA fill:#e2e8f0,stroke:#10b981,stroke-width:2px

    style UI fill:#eff6ff,stroke:#ef4444,stroke-width:2px
```

3.2. Cấu trúc chi tiết 4 tầng Project trong giải pháp Visual Studio

1. Tầng `KhoBanLe.CoreBusiness` (Tâm điểm kiến trúc - Lõi nghiệp vụ)

- **Vị trí tệp:** `KhoBanLe/KhoBanLe.CoreBusiness/KhoBanLe.CoreBusiness.csproj`
- **Đặc điểm:** Không chứa bất kỳ thẻ `<ProjectReference>` nào. Không phụ thuộc ASP.NET Core, không phụ thuộc thư viện cơ sở dữ liệu (Entity Framework, Dapper hay SqlClient).
- **Thành phần:**
    - `Product.cs`: Định nghĩa thực thể hàng hóa, đơn giá, tồn kho, định mức tối thiểu.
    - `Supplier.cs`, `Customer.cs`: Nhà cung cấp hàng hóa và khách hàng tích điểm.
    - `GoodsReceipt.cs` & `GoodsReceiptLine.cs`: Cặp thực thể Header - Line phiếu nhập.
    - `GoodsIssue.cs` & `GoodsIssueLine.cs`: Cặp thực thể Header - Line phiếu xuất.

- `PosOrder.cs` & `PosOrderLine.cs`: Cặp thực thể hóa đơn thanh toán tại quầy.
- `InventoryReportItem.cs`: Chứa công thức bất biến của Luật 2: \(\text{EndingStock} = \text{BeginningStock} + \text{TotalImport} - \text{TotalExport}\)
- `ReceiptStatus.cs`: Kiểu liệt kê trạng thái chứng từ (`Draft = 0`, `Approved = 1`, `Cancelled = 2`).

2. Tầng `KhoBanLe.UseCases` (Tầng điều phối ca sử dụng)

- **Vị trí tệp:** `KhoBanLe/KhoBanLe.UseCases/KhoBanLe.UseCases.csproj`
- **Đặc điểm: Chỉ tham chiếu duy nhất `KhoBanLe.CoreBusiness`**. Nơi định nghĩa các bản hợp đồng giao tiếp dữ liệu (Interface Repositories).
- **Thành phần:**
  - **Interface DataStore:** `IProductRepository`, `ISupplierRepository`, `ICustomerRepository`, `IGoodsReceiptRepository`, `IGoodsIssueRepository`, `IPosOrderRepository`, `IInventoryReportRepository`.
  - **Use Cases nghiệp vụ:**
    - `ApproveGoodsIssueUseCase.cs`: **Cài đặt LUẬT 1 (Dòng 72 - 90)**.
    - `GetInventoryReportUseCase.cs`: **Cài đặt LUẬT 2 (Dòng 42 - 59)**.
    - `CreateGoodsReceiptUseCase.cs`, `CreateGoodsIssueUseCase.cs`, `CreatePosOrderUseCase.cs`.
    - `ParseSupplierQuoteUseCase.cs`: Module AI SmartPaste trích xuất báo giá.

3. Tầng `KhoBanLe.DataStore.HardCoded` (Tầng Plugin / Hạ tầng dữ liệu)

- **Vị trí tệp:** `KhoBanLe/KhoBanLe.DataStore.HardCoded/KhoBanLe.DataStore.HardCoded.csproj`
- **Đặc điểm:** Tham chiếu `CoreBusiness` và `UseCases`. Là nơi hiện thực hóa (implement) các Interface Repository.
- **Thành phần:**
  - Thư mục `Sql/`: Chứa các Repository kết nối Microsoft SQL Server qua `Microsoft.Data.SqlClient` và `Dapper`:
    - `SqlProductRepository.cs`: Truy vấn danh mục, tìm kiếm, cập nhật tồn.

        - `SqlGoodsReceiptRepository.cs`: Ghi Header-Line và Phê duyệt tăng tồn trong 1 Transaction (`connection.BeginTransaction()`).
        - `SqlGoodsIssueRepository.cs`: Ghi Header-Line và Phê duyệt trừ tồn trong 1 Transaction.
        - `SqlPosOrderRepository.cs`: Xử lý thanh toán đơn hàng POS, trừ tồn và sinh phiếu xuất tự động trong 1 Transaction.
        - `SqlInventoryReportRepository.cs`: Truy vấn đối soát tồn kho tham số hóa.
    - Thư mục gốc: Chứa các lớp Repository In-Memory (giữ danh sách dữ liệu trong RAM) phục vụ kiểm thử đơn vị độc lập.

4. Tầng `KhoBanLe` (Tầng Ứng dụng Web / Composition Root)

- **Vị trí tệp:** `KhoBanLe/KhoBanLe/KhoBanLe.csproj`
- **Đặc điểm:** Tham chiếu cả 3 project trên để đóng vai trò **Composition Root** — nơi lắp ráp các phụ thuộc tại `Program.cs`.
- **Thành phần:**
    - `Program.cs`: Đăng ký Dependency Injection cho Repositories, UseCases và State Services.
    - `Components/Pages/`: Các trang giao diện Blazor Server tương tác người dùng:
        - `Products/ProductList.razor`: Quản lý danh mục hàng hóa, EditForm có DataAnnotations (K3.2).
        - `Cart/PosCashier.razor`: Giao diện quầy bán lẻ POS cảm ứng, tích hợp giỏ hàng Scoped và in bill nhiệt.
        - `Receipts/CreateReceipt.razor`: Lập phiếu nhập kho, tích hợp Modal AI SmartPaste.
        - `Issues/CreateIssue.razor`: Lập phiếu xuất kho.
        - `Manager/ApprovalList.razor`: Màn hình duyệt phiếu xuất/nhập, kiểm tra phân quyền URL và demo Luật 1.
        - `Reports/InventoryReport.razor`: Báo cáo đối soát Nhập - Xuất - Tồn, xuất Excel thật UTF-8 BOM.
        - `Guide/CapstoneGuide.razor`: Bảng kiểm soát trực quan 17 mã phiếu chấm và cẩm nang vấn đáp.
    - `Components/Controls/`: Thư viện linh kiện tái sử dụng có `[Parameter]` và `EventCallback` (K3.1): `LineItemEditor.razor`, `ProductSearchBar.razor`, `ConfirmActionModal.razor`, `StockStatusBadge.razor`.

- `Services/`: Các dịch vụ ứng dụng: `ShoppingCartService.cs`, `DraftReceiptStateService.cs`, `UserRoleService.cs`, `ExcelExportService.cs`.

3.3. Cơ chế hoán đổi Data Plugin (Swappable Plugin: SQL Server & In-Memory)

Một trong những điểm ưu việt nhất của Clean Architecture được giảng viên đánh giá rất cao trong kỳ thi vấn đáp (K5.2) là khả năng hoán đổi cơ chế lưu trữ dữ liệu mà không làm thay đổi dù chỉ 1 dòng mã nguồn trong tầng nghiệp vụ (`UseCases`) hay tầng lõi (`CoreBusiness`).

Tại tệp Program.cs, việc đổi nguồn dữ liệu từ SQL Server sang In-Memory (hoặc ngược lại) được cấu hình tập trung:

```
// Đọc cấu hình từ appsettings.json

bool useSqlDataStore = builder.Configuration.GetValue<bool>("UseSqlDataStore", tru
e);



if (useSqlDataStore)

{

    // [PLUGIN SQL SERVER - DAPPER]: Đáp ứng K2.2, K2.3, K2.4

    builder.Services.AddScoped<IProductRepository, SqlProductRepository>();

    builder.Services.AddScoped<ISupplierRepository, SqlSupplierRepository>();

    builder.Services.AddScoped<IGoodsReceiptRepository, SqlGoodsReceiptRepository>
();

    builder.Services.AddScoped<IGoodsIssueRepository, SqlGoodsIssueRepository>();

    builder.Services.AddScoped<IInventoryReportRepository, SqlInventoryReportRepos
itory>();

    builder.Services.AddScoped<ICustomerRepository, SqlCustomerRepository>();

    builder.Services.AddScoped<IPosOrderRepository, SqlPosOrderRepository>();

}

else

{

    // [PLUGIN IN-MEMORY]: Đáp ứng chạy offline mock data

    builder.Services.AddSingleton<IProductRepository, ProductRepository>();

    builder.Services.AddSingleton<ISupplierRepository, SupplierRepository>();

    builder.Services.AddSingleton<IGoodsReceiptRepository, GoodsReceiptRepository>
();
```

```
    builder.Services.AddSingleton<IGoodsIssueRepository, GoodsIssueRepository>();

    builder.Services.AddSingleton<IInventoryReportRepository, InventoryReportRepos
itory>();

    builder.Services.AddSingleton<ICustomerRepository, CustomerRepository>();

    builder.Services.AddSingleton<IPosOrderRepository, PosOrderRepository>();

}
```

3.4. Quản lý trạng thái và vòng đời Service (Scoped, Transient, Singleton)

Hệ thống tuân thủ chặt chẽ nguyên tắc phân bổ vòng đời dịch vụ (Service Lifetime) của ASP.NET Core:

- `AddTransient`: Dành cho tất cả các **UseCases** nghiệp vụ (`ISearchProductsUseCase`, `IApproveGoodsIssueUseCase`...). Mỗi khi một component yêu cầu thực thi, một thể hiện mới được cấp phát và giải phóng ngay sau đó, đảm bảo không lưu vết trạng thái thừa giữa các lần gọi.
- `AddScoped`: Cực kỳ quan trọng trong Blazor Server. Service tồn tại xuyên suốt vòng đời một phiên kết nối SignalR Circuit của người dùng. Áp dụng cho:
    - `DraftReceiptStateService`: Giữ danh sách mặt hàng đang lập dở của phiếu nhập.
    - `ShoppingCartService`: Giữ giỏ hàng bán lẻ của thu ngân quầy POS.
    - `UserRoleService`: Giữ danh tính đăng nhập và vai trò người dùng trong phiên làm việc.
    - Các Repository SQL: Đảm bảo DbConnection và DbTransaction đồng bộ trong phạm vi một yêu cầu.
- `AddSingleton`: Áp dụng cho `ISqlConnectionFactory` để tái sử dụng cấu hình chuỗi kết nối duy nhất trong suốt vòng đời ứng dụng.

CHƯƠNG 4: THIẾT KẾ CƠ SỞ DỮ LIỆU & QUAN HỆ THỰC THỂ (ERD)

4.1. Chuẩn hóa dữ liệu 3NF & Cấu trúc 9 bảng dữ liệu

Cơ sở dữ liệu mang tên `KhoBanLe` được xây dựng trên Microsoft SQL Server, đạt chuẩn hóa dạng 3NF (Third Normal Form) nhằm loại bỏ hoàn toàn dư thừa dữ liệu và dị thường khi cập nhật:

```
erDiagram
```

```
    Supplier ||--o{ GoodsReceipt : "1 cung cấp nhiều"
    GoodsReceipt ||--|{ GoodsReceiptLine : "1 header nhiều line"
    Product ||--o{ GoodsReceiptLine : "nằm trong"

    Customer ||--o{ PosOrder : "mua nhiều đơn"
    PosOrder ||--|{ PosOrderLine : "1 header nhiều line"
    Product ||--o{ PosOrderLine : "nằm trong"

    GoodsIssue ||--|{ GoodsIssueLine : "1 header nhiều line"
    Product ||--o{ GoodsIssueLine : "nằm trong"

    PosOrder |o--o| GoodsIssue : "liên kết 1-1 trừ kho"

    Supplier {
        int Id PK
        string SupplierCode UK
        string Name
        string Phone
        string Address
    }
    Product {
        int Id PK
        string Sku UK
        string Name
        string Unit
        string Category
        decimal Price
        int CurrentStock
        int MinStockLevel
    }
    Customer {
```

```
        int Id PK
        string CustomerCode UK
        string Name
        string Phone
        int Points
    }
    GoodsReceipt {
        int Id PK
        string ReceiptNumber UK
        int SupplierId FK
        datetime ReceiptDate
        int Status
    }
    GoodsReceiptLine {
        int Id PK
        int GoodsReceiptId FK
        int ProductId FK
        int Quantity
        decimal UnitPrice
        decimal LineTotal
    }
    GoodsIssue {
        int Id PK
        string IssueNumber UK
        string CustomerOrDepartment
        datetime IssueDate
        int Status
        int PosOrderId FK
    }
    GoodsIssueLine {
        int Id PK
```

```
        int GoodsIssueId FK
        int ProductId FK
        int Quantity
        decimal UnitPrice
        decimal LineTotal
    }
    PosOrder {
        int Id PK
        string OrderNumber UK
        int CustomerId FK
        decimal TotalAmount
        decimal ReceivedAmount
        decimal ChangeAmount
        int GoodsIssueId FK
        string Status
    }
    PosOrderLine {
        int Id PK
        int PosOrderId FK
        int ProductId FK
        int Quantity
        decimal UnitPrice
        decimal LineTotal
    }
```

4.2. Mối quan hệ khóa ngoại (10 Foreign Keys) & Toàn vẹn tham chiếu

Cơ sở dữ liệu thiết lập **10 ràng buộc Foreign Key** tường minh, bảo đảm tính toàn vẹn dữ liệu ở cấp độ Engine lưu trữ:

1. `FK_GoodsReceipt_Supplier`: Liên kết `GoodsReceipt.SupplierId` → `Supplier.Id`.
2. `FK_ReceiptLine_Receipt`: Liên kết `GoodsReceiptLine.GoodsReceiptId` → `GoodsReceipt.Id` (`ON DELETE CASCADE`).

3. `FK_ReceiptLine_Product`: Liên kết `GoodsReceiptLine.ProductId` → `Product.Id`.
4. `FK_IssueLine_Issue`: Liên kết `GoodsIssueLine.GoodsIssueId` → `GoodsIssue.Id` (`ON DELETE CASCADE`).
5. `FK_IssueLine_Product`: Liên kết `GoodsIssueLine.ProductId` → `Product.Id`.
6. `FK_PosOrder_Customer`: Liên kết `PosOrder.CustomerId` → `Customer.Id`.
7. `FK_PosOrderLine_PosOrder`: Liên kết `PosOrderLine.PosOrderId` → `PosOrder.Id` (`ON DELETE CASCADE`).
8. `FK_PosOrderLine_Product`: Liên kết `PosOrderLine.ProductId` → `Product.Id`.
9. `FK_GoodsIssue_PosOrder`: Liên kết `GoodsIssue.PosOrderId` → `PosOrder.Id`.
10. `FK_PosOrder_GoodsIssue`: Liên kết `PosOrder.GoodsIssueId` → `GoodsIssue.Id`.

Note

Mối liên kết chéo giữa `PosOrder` và `GoodsIssue` là thiết kế xuất sắc giúp hệ thống đồng bộ 100% giữa hoạt động bán lẻ tại quầy và hoạt động kế toán kho. Mỗi đơn hàng POS sau khi thanh toán đều gắn liền với 1 mã phiếu xuất tương ứng.

4.3. Computed Columns, Ràng buộc CHECK và Chỉ mục tối ưu (Indexes)

- **Cột tính toán lưu trữ (`PERSISTED Computed Column`)**: Tại các bảng chi tiết `GoodsReceiptLine`, `GoodsIssueLine`, `PosOrderLine`, thuộc tính `LineTotal` được định nghĩa: `LineTotal AS (Quantity * UnitPrice) PERSISTED` Cột này được SQL Server tự động tính và lưu trữ vật lý, giúp tăng tốc tối đa các câu lệnh tính tổng doanh thu/chi phí mà không cần tính toán lại qua CPU khi truy vấn.
- **Ràng buộc CHECK dữ liệu hợp lệ**:
    - `CHECK (CurrentStock >= 0)` trong bảng `Product`: Đảm bảo cấp độ DB không bao giờ chấp nhận số tồn âm (Phòng thủ đa tầng bổ trợ cho Luật 1).
    - `CHECK (Price >= 0)` và `CHECK (Quantity > 0)`: Ngăn chặn nhập đơn giá âm hoặc số lượng bằng 0.
- **Hệ thống Chỉ mục (Nonclustered Indexes)**: Tạo index trên các cột thường xuyên tìm kiếm và lọc: `IX_Product_Sku`, `IX_Product_Category`, `IX_Receipt_Status`, `IX_Receipt_Date`, `IX_Issue_Status`, `IX_PosOrder_Number`, `IX_PosOrder_Date`.

4.4. Các Views báo cáo và Stored Procedure bán lẻ POS trong 1 Transaction

Tệp 03_pos_queries_and_diagram.sql cung cấp sẵn các đối tượng SQL nâng cao:

1. **View** `v_PosOrderDetails`: Kết hợp thông tin đơn hàng POS, chi tiết sản phẩm, thông tin khách hàng và số phiếu xuất tương ứng.
2. **View** `v_DailyPosRevenue`: Thống kê doanh thu, tổng số đơn và phân bổ doanh thu theo hình thức thanh toán (Tiền mặt, VietQR, Thẻ) theo từng ngày.
3. **Stored Procedure** `sp_ProcessPosCheckout`: Gói gọn toàn bộ chu trình thanh toán bán lẻ trong một khối `BEGIN TRANSACTION ... COMMIT TRANSACTION`, xử lý từ khâu kiểm tra tồn kho → Trừ tồn sản phẩm → Sinh phiếu xuất → Lưu đơn POS.

CHƯƠNG 5: CÀI ĐẶT 2 LUẬT NGHIỆP VỤ CỐT LÕI (CORE BUSINESS RULES)

Đây là phần trọng tâm quyết định điểm số cao của đồ án (tiêu chí K1.3 và K3.3). Cả hai luật đều được cài đặt tại tầng `UseCases` và `CoreBusiness`, tuyệt đối không viết bừa bãi trong các tệp giao diện `.razor`.

5.1. LUẬT 1: Duyệt mới đổi tồn & Chặn xuất âm kho (K1.3, K3.3)

Nội dung quy tắc nghiệp vụ:

1. Khi nhân viên kho tạo phiếu xuất kho (`GoodsIssue`), phiếu ở trạng thái Chờ duyệt (`Status = Draft = 0`). **Tồn kho thực tế của hàng hóa trong bảng Product hoàn toàn KHÔNG bị thay đổi.**
2. Hàng hóa chỉ bị trừ khỏi kho khi **Quản lý kho** thực hiện hành động **Phê duyệt (`Status = Approved = 1`).**
3. Trước khi thực thi việc trừ kho, hệ thống **bắt buộc kiểm tra từng dòng hàng hóa**. Nếu phát hiện bất kỳ dòng hàng nào có số lượng yêu cầu xuất lớn hơn số lượng tồn kho khả dụng (\(CurrentStock < Quantity\)), hệ thống **BẬT CƠ CHẾ CHẶN NGAY LẬP TỨC**:
    - Không cho phép phê duyệt.
    - Không trừ kho bất kỳ sản phẩm nào.
    - Trả về thông điệp cảnh báo thân thiện hiển thị trên giao diện (Alert màu đỏ), không văng lỗi unhandled exception (K3.3).
4. Nếu tất cả các dòng hàng đều hợp lệ (\(CurrentStock \ge Quantity\)), việc đổi trạng thái phiếu xuất và trừ tồn kho các mặt hàng phải được thực thi trong **1 Database Transaction duy nhất** (K2.2).

Đoạn mã minh chứng cài đặt Luật 1:

Vị trí tệp: ApproveGoodsIssueUseCase.cs

```
// KIỂM TRA LUẬT 1: Chặn xuất âm kho trên từng dòng hàng hóa
```

```
foreach (var line in issue.Lines)

{

    var product = await _productRepository.GetProductByIdAsync(line.ProductId);

    if (product == null)

    {

        return ApproveGoodsIssueResult.Fail($"Mặt hàng mã #{line.ProductId} không
tồn tại trong hệ thống.");

    }



    if (product.CurrentStock < line.Quantity)

    {

        // Chặn lại ngay lập tức, không trừ kho, trả về thông báo lỗi thân thiện c
huẩn Slide 7

        return ApproveGoodsIssueResult.Fail(

            $"Hàng {product.Name} chỉ còn tồn {product.CurrentStock}, không đủ để
xuất {line.Quantity}!",

            product.Name,

            product.CurrentStock,

            line.Quantity);

    }

}



// Tất cả các dòng hợp lệ -> Tiến hành duyệt & trừ kho trong 1 transaction an toàn

await _goodsIssueRepository.ApproveIssueTransactionAsync(issueId, approvedBy);



return ApproveGoodsIssueResult.Ok();
```

Ca kiểm thử vi phạm chuẩn bị sẵn (Ready-to-Demo Case):

- Trong CSDL mẫu (`02_seed_data.sql`), hệ thống chuẩn bị sẵn phiếu xuất **`PX-2026-002`**:
    - Yêu cầu xuất: **10 chiếc** sản phẩm *Bút bi Thiên Long TL-027*.
    - Tồn kho thực tế trong kho: chỉ còn **5 chiếc**.

- Khi Quản lý kho truy cập trang `/approvals` và nhấn nút **[Duyệt Xuất]**, hệ thống ngay lập tức kích hoạt Luật 1, chặn đứng thao tác và hiển thị banner đỏ nổi bật:

> *"Hàng Bút bi Thiên Long TL-027 chỉ còn tồn 5, không đủ để xuất 10!"*

- Mở SQL Server SSMS kiểm tra bảng `Product`, tồn kho của mặt hàng vẫn giữ nguyên vẹn là 5.

5.2. LUẬT 2: Bảo toàn công thức Nhập - Xuất - Tồn & Đối soát chéo sổ cái (K1.3, K4.3)

Nội dung quy tắc nghiệp vụ:

1. **Công thức bất biến kế toán kho**: $\text{Tồn cuối kỳ} = \text{Tồn đầu kỳ} + \sum \text{Nhập đã duyệt (Approved)} - \sum \text{Xuất đã duyệt (Approved)}$
2. **Khóa chứng từ**: Mọi phiếu nhập và phiếu xuất khi đã ở trạng thái `Approved` đều bị khóa cứng, không cho phép chỉnh sửa số lượng hay xóa bỏ, bảo đảm tính toàn vẹn lịch sử.
3. **Cơ chế đối soát chéo độc lập (Audit Check)**: Hệ thống so sánh độc lập giữa:
    - Con số EndingStock tính toán từ tổng các dòng chứng từ giao dịch thực tế đã phát sinh.
    - Con số CurrentStock được ghi nhận tại bảng hàng hóa `Product`. Nếu $\text{EndingStock} == \text{CurrentStock}$, hệ thống ghi nhận trạng thái **Cân bằng (`IsBalanced = true`)**. Nếu có sai lệch, hệ thống lập tức phát cảnh báo đỏ và liệt kê danh sách chênh lệch.

Đoạn mã minh chứng cài đặt Luật 2:

- Thuộc tính tính toán bất biến tại InventoryReportItem.cs:

```
// Công thức bất biến: Tồn cuối kỳ = Tồn đầu kỳ + Tổng nhập - Tổng xuất

public int EndingStock => BeginningStock + TotalImport - TotalExport;


// Tồn thực tế trong bảng Product để kiểm tra đối soát chéo độc lập

public int CurrentStock { get; set; }


// Kiểm tra đối soát: Kết quả tính toán từ chứng từ có khớp 100% với tồn kho thực tế?

public bool IsBalanced => EndingStock == CurrentStock;
```

- Hàm thực thi kiểm tra đối soát tại GetInventoryReportUseCase.cs:

```
public async Task<InventoryAuditReport> ExecuteAsync(DateTime? fromDate = null, Da
teTime? toDate = null)

{

    var items = (await _reportRepository.GetInventoryReportAsync(fromDate, toDate)
).ToList();



    var discrepancies = items.Where(i => !i.IsBalanced).ToList();

    bool passed = !discrepancies.Any();



    string summaryMessage = passed

        ? "Đối soát thành công: 100% mặt hàng bảo toàn chuẩn công thức Nhập - Xuất
- Tồn và khớp hoàn toàn với sổ cái kho."

        : $"CẢNH BÁO ĐỐI SOÁT: Phát hiện {discrepancies.Count} mặt hàng có chênh l
ệch giữa chứng từ giao dịch và tồn kho thực tế!";



    return new InventoryAuditReport

    {

        Items = items,

        IsAuditPassed = passed,

        DiscrepancyCount = discrepancies.Count,

        AuditSummaryMessage = summaryMessage

    };

}
```

5.3. Đảm bảo tính toàn vẹn giao dịch Header-Line qua 1 Database Transaction (K2.2)

Tất cả các hành động ghi dữ liệu phức hợp có liên quan giữa bảng Header và bảng Line đều được bọc trong khối giao dịch `connection.BeginTransaction()` của `Microsoft.Data.SqlClient`.

Minh chứng tại SqlGoodsReceiptRepository.cs và SqlGoodsIssueRepository.cs:

```
using var connection = _connectionFactory.CreateConnection();

await connection.OpenAsync();

using var transaction = connection.BeginTransaction();
```

```
try

{

    // 1. Cập nhật trạng thái phiếu xuất sang Approved

    const string sqlApprove = @"

        UPDATE dbo.GoodsIssue

        SET Status = 1, ApprovedBy = @ApprovedBy, ApprovedDate = GETDATE()

        WHERE Id = @IssueId;";

    await connection.ExecuteAsync(sqlApprove, new { IssueId = issueId, ApprovedBy
= approvedBy }, transaction);



    // 2. Trừ tồn kho từng sản phẩm

    foreach (var line in issue.Lines)

    {

        const string sqlUpdateStock = @"

            UPDATE dbo.Product

            SET CurrentStock = CurrentStock - @Quantity

            WHERE Id = @ProductId;";

        await connection.ExecuteAsync(sqlUpdateStock, new { line.Quantity, line.Pr
oductId }, transaction);

    }



    // Cam kết hoàn tất giao dịch ACID

    transaction.Commit();

}

catch

{

    // Tự động Rollback nếu xảy ra lỗi bất kỳ

    transaction.Rollback();

    throw;

}
```

CHƯƠNG 6: CHI TIẾT CÁC PHÂN HỆ CHỨC NĂNG CỦA HỆ THỐNG

6.1. Phân hệ Bán hàng tại quầy (POS) & In Bill nhiệt K80

- **Giao diện cảm ứng hiện đại**: Thiết kế tối ưu cho màn hình cảm ứng POS hoặc máy tính bàn của thu ngân. Danh mục hàng hóa chia tab phân loại trực quan (Tạp hóa, Đồ uống, Đồ ăn vặt, Văn phòng phẩm).
- **Tìm kiếm thần tốc**: Hỗ trợ tìm kiếm theo tên hoặc mã SKU/Barcode tức thời, thêm vào giỏ bằng 1 chạm hoặc phím tắt.
- **Quản lý giỏ hàng Scoped**: Dịch vụ `ShoppingCartService` quản lý các mặt hàng trong giỏ, tự động cập nhật số lượng, đơn giá và tính thành tiền tức thì.
- **Thanh toán đa phương thức**:
    1. *Tiền mặt*: Nhập số tiền khách đưa → Hệ thống tự động tính số tiền thừa trả lại chính xác.
    2. *Chuyển khoản VietQR*: Hệ thống tự động sinh mã VietQR chuẩn NAPAS 247 có nhúng sẵn số tài khoản ngân hàng, tên người nhận và đúng số tiền cần thanh toán theo từng đơn hàng.
    3. *Quẹt thẻ ngân hàng (POS Card)*: Hỗ trợ tích hợp quẹt thẻ tín dụng/ATM.
- **In hóa đơn nhiệt chuẩn khổ K80**: Sau khi bấm Thanh toán, hệ thống mở modal in bill nhiệt K80 chuẩn mực bao gồm: Tên siêu thị, Địa chỉ, Số hóa đơn, Ngày giờ, Chi tiết từng món, Tổng tiền, Tiền khách đưa, Tiền thối và lời cảm ơn.

6.2. Phân hệ Quản lý Nhập kho & Trợ lý AI SmartPaste (Mục 10)

- **Lập phiếu nhập tiêu chuẩn**: Cho phép chọn Nhà cung cấp, nhập ngày nhập, ghi chú và thêm nhiều dòng hàng thông qua component dùng lại `LineItemEditor.razor`.
- **Trợ lý AI SmartPaste Báo Giá Nhà Cung Cấp (Đạt chuẩn 5 tiêu chí Mục 10)**:
    - *Tiêu chuẩn 1 (Bảo mật API Key)*: Được cấu hình qua User Secrets/Biến môi trường, không đưa lên GitHub.
    - *Tiêu chuẩn 2 (Độc lập kiến trúc)*: AI được trừu tượng hóa và inject vào UseCase riêng biệt `ParseSupplierQuoteUseCase`, không gọi trực tiếp từ `.razor`.
    - *Tiêu chuẩn 3 (Xử lý lỗi mềm)*: Khi mất mạng hoặc không có API key, bộ phân tích Heuristic thông minh dự phòng vẫn tự động nhận

diện mẫu văn bản tiếng Việt để trích xuất số lượng, tên hàng và đơn giá chuẩn xác.

- *Tiêu chuẩn 4 (Human-in-the-loop)*: AI chỉ đề xuất dữ liệu. Màn hình hiển thị bảng xem trước (Preview Modal) để Thủ kho kiểm tra, chỉnh sửa trước khi xác nhận đưa vào phiếu nhập chính thức.
- *Tiêu chuẩn 5 (Chi phí 0 đồng)*: Hỗ trợ Ollama cục bộ hoặc Regex Heuristic Engine không tốn bất kỳ chi phí bản quyền nào.

6.3. Phân hệ Quản lý Xuất kho & Kiểm soát phiếu

- Phục vụ các nhu cầu xuất kho bán buôn, xuất điều chuyển nội bộ hoặc xuất hủy hàng hỏng.
- Quản lý quy trình kiểm duyệt chặt chẽ: Phiếu mới tạo luôn ở trạng thái `Draft`. Chỉ Quản lý kho mới có quyền bấm duyệt xuất sau khi hệ thống đã kiểm tra thỏa mãn Luật 1.

6.4. Phân hệ Quản lý Danh mục hàng hóa & Cảnh báo tồn kho tối thiểu

- Quản lý danh mục hơn 29 mặt hàng bán lẻ phổ biến (Mì Hảo Hảo, Nước ngọt Coca/Pepsi, Bánh snack O'Star, Gạo ST25, Sữa Vinamilk, Văn phòng phẩm...).
- Hiển thị huy hiệu trực quan (`StockStatusBadge`):
  - Màu xanh: Tồn kho dồi dào.
  - Màu vàng (Low Stock Alert): Tồn kho xuống dưới mức định mức an toàn (`CurrentStock <= MinStockLevel`), nhắc nhở thủ kho kịp thời nhập thêm hàng.
  - Màu đỏ: Hết hàng.
- Form thêm/sửa sản phẩm sử dụng `EditForm` kết hợp `DataAnnotations` bắt lỗi dữ liệu ngay tại ô nhập.

6.5. Phân hệ Báo cáo Thống kê NXT & Xuất dữ liệu thật ra Excel

- Lọc báo cáo Nhập - Xuất - Tồn theo khoảng thời gian tùy chọn (Từ ngày ... Đến ngày ...).
- Hiển thị bảng đối soát chi tiết từng mặt hàng: Tồn đầu kỳ, Tổng nhập, Tổng xuất, Tồn cuối kỳ tính toán và Tồn kho thực tế sổ cái.
- **Xuất tệp Excel / CSV chuẩn UTF-8 BOM**: Thông qua `ExcelExportService`, hệ thống xuất dữ liệu thực tế từ SQL Server ra tệp bảng tính có hỗ trợ hiển thị tiếng Việt có dấu chuẩn xác 100% khi mở trực tiếp bằng Microsoft Excel.

6.6. Phân hệ Bảo mật, Phân quyền & Phòng thủ URL (K4.1, K4.2)

- Thanh công cụ Header hỗ trợ chuyển đổi linh hoạt tài khoản giữa Thủ kho (`WarehouseStaff`) và Quản lý kho (`InventoryManager`).
- **Cơ chế phòng thủ URL gõ thẳng (K4.2)**: Khi đang đăng nhập với vai trò Thủ kho, nếu người dùng cố tình nhập đường dẫn quản trị `http://localhost:5000/approvals` trên thanh địa chỉ trình duyệt, hệ thống sẽ chặn đứng truy cập và hiển thị màn hình `403 - ACCESS DENIED` với thông báo bảo mật rõ ràng.

CHƯƠNG 7: BẢNG ĐỐI CHIẾU 17 MÃ KIỂM TRA PHIẾU CHẤM (K1.1 – K5.3)

Toàn bộ đồ án được thiết kế chuẩn xác theo **Mục 12: Phiếu chấm chi tiết** trong tài liệu hướng dẫn Capstone của Bộ môn, cam kết đạt điểm tuyệt đối **100 / 100 điểm**:

Mã	Nội Dung Tiêu Chí Đánh Giá	Điểm	Vị Trí Tệp Tin & Dòng Code Làm Bằng Chứng Cụ Thể	Đánh Giá
K1.1	Đủ 4 project; CoreBusiness không tham chiếu ai; UseCases chỉ tham chiếu CoreBusiness	**8đ**	• `KhoBanLe.CoreBusiness.csproj` (0 ProjectReference) • `KhoBanLe.UseCases.csproj` (chỉ tham chiếu CoreBusiness) • `KhoBanLe.DataStore.HardCoded.csproj` (ref Core + UseCases) • `KhoBanLe.csproj` (Composition Root)	**Đạt đủ (8/8)**
K1.2	Interface repo nằm ở UseCases, Plugin cài đặt; .razor chỉ inject interface	**7đ**	• Interface: `KhoBanLe.UseCases/PluginInterfaces/DataStore/` • Cài đặt: `KhoBanLe.DataStore.HardCoded/Sql/` • File .razor: `ProductList.razor`, `ApprovalList.razor` (chỉ inject UseCases & Interfaces)	**Đạt đủ (7/7)**
K1.3	2 luật nghiệp vụ của đề cài trong CoreBusiness / UseCases (3đ/luật)	**6đ**	• **Luật 1 (Chặn xuất âm):** `ApproveGoodsIssueUseCase.cs` (Dòng 72-90) • **Luật 2 (Bảo toàn NXT):** `GetInventoryReportUseCase.cs` (Dòng 42-59) & `InventoryReportItem.cs` (Dòng 19-26)	**Đạt đủ (6/6)**

Mã	Nội Dung Tiêu Chí Đánh Giá	Điểm	Vị Trí Tệp Tin & Dòng Code Làm Bằng Chứng Cụ Thể	Đánh Giá
K1.4	DI đăng ký đủ; giỏ/lựa chọn tạm dùng `Scoped`	**4đ**	• `Program.cs` (Dòng 102-105): `DraftReceiptStateService` (Scoped), `ShoppingCartService` (Scoped), `UserRoleService` (Scoped)	**Đạt đủ (4/4)**
K2.1	Từ 5 bảng, 3NF, có khóa ngoại; script tạo bảng + seed data chạy trên CSDL trống	**8đ**	• `SqlScripts/01_schema.sql` (9 bảng 3NF, 10 FK, Indexes) • `SqlScripts/02_seed_data.sql` (Dữ liệu mẫu đầy đủ, chuẩn logic) • `SqlScripts/03_pos_queries_and_diagram.sql` (Views, SP bán hàng POS)	**Đạt đủ (8/8)**
K2.2	Cặp Header – Line ghi trong 1 Database Transaction	**6đ**	• `SqlGoodsReceiptRepository.cs` (Dòng 74-131) • `SqlGoodsIssueRepository.cs` (Dòng 70-123) • `SqlPosOrderRepository.cs` (Dòng 66-160) Tất cả đều dùng khối `connection.BeginTransaction()`	**Đạt đủ (6/6)**
K2.3	Mọi truy vấn tham số hóa, không nối chuỗi SQL	**5đ**	100% câu truy vấn trong `KhoBanLe.DataStore.HardCoded/Sql/` đều dùng tham số `@Param` của Dapper, không dùng `$"SELECT` hay `+ "WHERE`	**Đạt đủ (5/5)**
K2.4	Luồng giao dịch chính chạy end-to-end và dữ liệu nằm trong SQL Server	**6đ**	Kết nối trực tiếp CSDL `KhoBanLe` trên localhost qua `appsettings.json`. Thao tác lập phiếu, duyệt kho, thanh toán POS ghi trực tiếp vào SSMS	**Đạt đủ (6/6)**
K3.1	Ít nhất 3 component tái sử dụng có `[Parameter]` / `EventCallback`	**6đ**	1. `LineItemEditor.razor` (QuantityChanged, UnitPriceChanged, OnDelete) 2. `ProductSearchBar.razor` (SearchTermChanged, OnSearch) 3. `ConfirmActionModal.razor` (OnConfirm, OnCancel) 4. `StockStatusBadge.razor` (OnBadgeClick)	**Đạt đủ (6/6)**

Mã	Nội Dung Tiêu Chí Đánh Giá	Điểm	Vị Trí Tệp Tin & Dòng Code Làm Bằng Chứng Cụ Thể	Đánh Giá
K3.2	`EditForm` + `DataAnnotations`; lỗi nhập liệu hiển thị cạnh ô nhập	**5đ**	• `ProductList.razor` (Modal Thêm/Sửa mặt hàng) • `CreateReceipt.razor` (Form tạo phiếu nhập) • `CreateIssue.razor` (Form tạo phiếu xuất)	**Đạt đủ (5/5)**
K3.3	Ca vi phạm 2 luật nghiệp vụ hiện thông báo thân thiện, không văng lỗi	**4đ**	• `ApprovalList.razor` (Dòng 74-88): Bấm duyệt phiếu vi phạm `PX-2026-002` hiển thị Alert đỏ thân thiện, không crash ứng dụng	**Đạt đủ (4/4)**
K3.4	Dùng được trên màn hình điện thoại (bề rộng 390px)	**5đ**	`app.css` thiết kế responsive CSS Grid & Flexbox, bảng biểu tự chuyển đổi sang dạng thẻ Card View trên thiết bị di động bề rộng 390px	**Đạt đủ (5/5)**
K4.1	Đăng nhập, đăng xuất; ít nhất 2 vai trò	**4đ**	`UserRoleService.cs`: Hỗ trợ 2 vai trò `WarehouseStaff` và `InventoryManager`, chuyển đổi tài khoản trực tiếp trên Header	**Đạt đủ (4/4)**
K4.2	Tài khoản thường gõ thẳng URL trang quản trị bị chặn	**5đ**	`ApprovalList.razor` (Dòng 14-41): Tài khoản Thủ kho gõ URL `/approvals` bị chặn và hiển thị màn hình 403 Access Denied	**Đạt đủ (5/5)**
K4.3	Tính năng nâng cao: AI đạt đủ Mục 10, hoặc xuất báo cáo (Excel/PDF) từ dữ liệu thật	**6đ**	• AI SmartPaste: `ParseSupplierQuoteUseCase.cs` (5 tiêu chí Mục 10) • Xuất Excel: `ExcelExportService.cs` (Xuất file CSV/Excel UTF-8 BOM)	**Đạt đủ (6/6)**
K5.1	Use Case, Class, Sequence (PlantUML) khớp tên lớp và luồng trong code	**6đ**	• `Docs/Diagrams/usecase.puml` • `Docs/Diagrams/class.puml` (Khớp 100% 4 tầng Clean Architecture) • `Docs/Diagrams/sequence.puml` (Khớp luồng kiểm tra Luật 1 và Transaction)	**Đạt đủ (6/6)**

Mã	Nội Dung Tiêu Chí Đánh Giá	Điểm	Vị Trí Tệp Tin & Dòng Code Làm Bằng Chứng Cụ Thể	Đánh Giá
K5.2	Vấn đáp: mỗi thành viên chỉ đúng dòng cài luật và giải thích Dependency Rule	**6đ**	Cẩm nang chuẩn bị vấn đáp chi tiết tại Chương 8 của báo cáo này và tab Sẵn Sàng Vấn Đáp trên trang `/guide` của ứng dụng	**Đạt đủ (6/6)**
K5.3	Mọi thành viên có commit, rải trên ít nhất 3 tuần	**3đ**	Lịch sử Git: Sprint 1 (28/09), Sprint 2 (06/10), Sprint 3 (15/10 - 19/10), trải dài qua 4 tuần làm việc liên tục	**Đạt đủ (3/3)**
TỔNG	**TỔNG ĐIỂM DỰ KIẾN ĐẠT ĐƯỢC**	**100đ**	**ĐẠT ĐỦ 100 / 100 ĐIỂM (XẾP LOẠI XUẤT SẮC)**	**100/100**

CHƯƠNG 8: HƯỚNG DẪN CÀI ĐẶT, VẬN HÀNH & KỊCH BẢN DEMO BẢO VỆ

8.1. Hướng dẫn thiết lập môi trường và chạy dự án

Yêu cầu hệ thống:

- Hệ điều hành: Windows 10/11
- SDK: .NET 8.0 SDK (hoặc .NET 10.0 LTS)
- CSDL: Microsoft SQL Server (bản Express, Developer hoặc LocalDB)
- Công cụ quản lý CSDL: SQL Server Management Studio (SSMS) hoặc Azure Data Studio

Các bước thực hiện:

```
# Bước 1: Khởi tạo Cơ sở Dữ liệu

# Mở SQL Server Management Studio (SSMS), kết nối tới server localhost.

# Mở và lần lượt thực thi 3 script trong thư mục KhoBanLe/SqlScripts/:

#   1. 01_schema.sql (Tạo Database KhoBanLe và 9 bảng 3NF)

#   2. 02_seed_data.sql (Nạp dữ liệu mẫu ban đầu)
```

```
#   3. 03_pos_queries_and_diagram.sql (Tạo Views và Stored Procedure)


# Bước 2: Chuyển vào thư mục chứa WebApp

cd KhoBanLe/KhoBanLe


# Bước 3: Khởi chạy ứng dụng

dotnet run
```

Mở trình duyệt web (Google Chrome / Microsoft Edge) và truy cập vào địa chỉ hiển thị trên terminal: `http://localhost:5000` (hoặc cổng HTTPS tương ứng).

8.2. Kịch bản Demo 5 bước thuyết phục Hội đồng chấm thi

Khi báo cáo trước Giảng viên và Hội đồng, nhóm thực hiện theo kịch bản chuẩn sau:

1. **Bước 1: Giới thiệu Kiến trúc 4 Tầng & Quy tắc phụ thuộc (K1.1, K1.2)**
    - Mở Solution trên Visual Studio, chỉ rõ 4 project. Mở các file `.csproj` chứng minh `CoreBusiness` không tham chiếu ai, `UseCases` chỉ tham chiếu `CoreBusiness`.
    - Mở `Program.cs` chỉ rõ cách hoán đổi giữa SQL Server Plugin và In-Memory Plugin (K5.2).
2. **Bước 2: Demo Bán Hàng Tại Quầy POS & In Bill K80 (K2.2, K2.4)**
    - Đăng nhập với vai trò **Thủ kho**. Vào menu **Bán Lẻ POS** (`/cart/pos`).
    - Chọn vài sản phẩm (ví dụ: Mì Hảo Hảo, Coca Cola).
    - Chọn phương thức thanh toán Tiền mặt, nhập tiền khách đưa → Bấm **[Thanh Toán & In Hóa Đơn]**.
    - Màn hình hiển thị Bill nhiệt K80 chuyên nghiệp. Mở SSMS truy vấn bảng `PosOrder` và `Product` để chứng minh tồn kho đã tự động bị trừ ngay lập tức.
3. **Bước 3: Demo Trợ Lý AI SmartPaste Báo Giá NCC (K4.3 - Mục 10)**
    - Vào menu **Nhập Kho → Lập Phiếu Mới** (`/receipts/create`).
    - Nhấn nút **[AI SmartPaste Báo Giá]**. Dán đoạn văn bản:

> *"Giao 20 thùng mì Hảo Hảo giá 110k, 10 hộp sữa tươi Vinamilk giá 32k"*

- Bấm **Trích Xuất** → Dữ liệu tự động điền vào bảng xem trước. Bấm **Xác Nhận & Đưa Vào Phiếu** → Bấm Lưu phiếu nhập.

4. **Bước 4: Demo Ca Vi Phạm LUẬT 1 — Chặn Xuất Âm Kho (K1.3, K3.3)**
   - Chuyển vai trò sang **Quản Lý Kho**.
   - Vào trang **Duyệt Phiếu Xuất** (`/approvals`).
   - Chọn phiếu mẫu `PX-2026-002` (yêu cầu xuất 10 chiếc Bút bi Thiên Long, trong khi tồn chỉ có 5 chiếc).
   - Nhấn nút **[Duyệt Xuất]**.
   - **Kết quả:** Hệ thống lập tức chặn lại, hiển thị Alert đỏ thân thiện: *"Hàng Bút bi Thiên Long TL-027 chỉ còn tồn 5, không đủ để xuất 10!"*. Không văng exception, CSDL được bảo vệ 100%.
5. **Bước 5: Demo LUẬT 2 — Báo Cáo Đối Soát NXT & Xuất Excel (K1.3, K4.3)**
   - Vào menu **Báo Cáo NXT** (`/reports`).
   - Xem bảng đối soát: Cột Tồn cuối kỳ tính toán bằng chính xác Cột Tồn kho thực tế. Dòng thông báo màu xanh: *"Đối soát thành công: 100% mặt hàng bảo toàn chuẩn công thức..."*.
   - Bấm nút **[Xuất Báo Cáo Excel]** → Tải file `.csv` về mở trực tiếp bằng Excel, hiển thị đầy đủ tiếng Việt có dấu.

8.3. Bộ câu hỏi & Đáp án chuẩn bị vấn đáp tốt nghiệp (Oral Defense Q&A)

Câu hỏi 1: "Hãy chỉ chính xác dòng code cài đặt 2 luật nghiệp vụ của đề tài?"

> **Trả lời:**
>
> - **Luật 1 (Chặn xuất âm & Duyệt mới đổi tồn):** Cài đặt tại tệp `KhoBanLe.UseCases/GoodsIssues/ApproveGoodsIssueUseCase.cs`, dòng **72 đến 90**. Tại đây, hệ thống duyệt qua từng dòng hàng hóa, so sánh `product.CurrentStock < line.Quantity`. Nếu vi phạm, hàm lập tức trả về `ApproveGoodsIssueResult.Fail(...)` mà không chạm vào CSDL.
> - **Luật 2 (Bảo toàn công thức Tồn đầu + Nhập - Xuất = Tồn cuối):** Cài đặt tại tệp `KhoBanLe.CoreBusiness/InventoryReportItem.cs`, dòng **19 đến 26** định nghĩa thuộc tính `EndingStock => BeginningStock + TotalImport - TotalExport` và `IsBalanced => EndingStock == CurrentStock`; kết hợp hàm thẩm định đối soát tại `KhoBanLe.UseCases/Reports/GetInventoryReportUseCase.cs`, dòng **42 đến 59**.

Câu hỏi 2: "Giải thích nguyên tắc Dependency Rule và chứng minh cấu trúc của nhóm tuân thủ?"

> **Trả lời:** Nguyên tắc Dependency Rule phát biểu rằng: Mã nguồn ở tầng bên trong không bao giờ được phép phụ thuộc vào mã nguồn ở tầng bên ngoài. Trong dự án:
>
> 1. `CoreBusiness` là trung tâm, không có `<ProjectReference>` nào.

2. `UseCases` chỉ tham chiếu duy nhất `CoreBusiness`, định nghĩa các Interface Repository (`IProductRepository`, `IGoodsIssueRepository`...).
3. Tầng `DataStore.HardCoded` (Plugin hạ tầng) phụ thuộc ngược vào `UseCases` để cài đặt các Interface này.
4. Tầng `KhoBanLe` (WebApp) đóng vai trò là Composition Root, chỉ làm nhiệm vụ đăng ký DI tại `Program.cs`. Các trang giao diện `.razor` chỉ `@inject` Interface UseCase, không bao giờ inject lớp cụ thể hay Sql Connection.

Câu hỏi 3: "Làm thế nào để đổi Plugin dữ liệu sang hệ quản trị khác hoặc In-Memory?"

Trả lời: Nhờ áp dụng Dependency Inversion, việc đổi Plugin chỉ cần thay đổi tại tệp `Program.cs` ở dòng 32-56:

- Dùng SQL Server: Đăng ký `builder.Services.AddScoped<IProductRepository, SqlProductRepository>();`
- Dùng In-Memory: Đăng ký `builder.Services.AddSingleton<IProductRepository, ProductRepository>();` Toàn bộ logic nghiệp vụ trong `UseCases` và `CoreBusiness` được giữ nguyên vẹn 100% mà không phải biên dịch lại hay sửa một ký tự nào.

Câu hỏi 4: "Tại sao nhóm lại sử dụng Dapper Micro-ORM thay vì Entity Framework Core?"

Trả lời: Nhóm lựa chọn Dapper Micro-ORM vì 3 lý do chiến lược:

1. **Kiểm soát tuyệt đối câu lệnh SQL**: Dapper cho phép viết trực tiếp các câu truy vấn tối ưu, phục vụ hoàn hảo cho tiêu chí K2.3 (tham số hóa 100%) và K2.2 (quản lý Transaction tường minh qua `BeginTransaction`).
2. **Hiệu năng vượt trội**: Dapper gần như đạt tốc độ tương đương với ADO.NET thuần, loại bỏ chi phí theo dõi trạng thái đối tượng (Change Tracking) nặng nề của Entity Framework, cực kỳ phù hợp cho hệ thống bán lẻ POS đòi hỏi tốc độ xử lý tức thời.
3. **Phù hợp với kiến trúc Plugin**: Giúp việc bóc tách tầng DataStore thành một Plugin nhẹ nhàng, độc lập, không bị ràng buộc bởi DbContext của EF Core.

CHƯƠNG 9: KẾT LUẬN & HƯỚNG PHÁT TRIỂN

9.1. Kết quả đạt được

Sau thời gian nghiên cứu và triển khai nghiêm túc, dự án **WebKhoBanLe** đã hoàn thành xuất sắc tất cả các mục tiêu đề ra:

1. Xây dựng thành công ứng dụng quản lý kho và điểm bán lẻ POS hoàn chỉnh, hoạt động ổn định trên nền tảng .NET 8 Blazor Server và SQL Server.

2. Tuân thủ 100% nguyên tắc kiến trúc Clean Architecture 4 tầng, bảo đảm tính module hóa, dễ bảo trì và dễ mở rộng.
3. Cài đặt chuẩn mực 2 luật nghiệp vụ cốt lõi, bảo vệ dữ liệu toàn vẹn qua các Database Transaction ACID.
4. Tích hợp giao diện bán lẻ POS hiện đại, thanh toán VietQR động, in hóa đơn nhiệt K80 và trợ lý AI SmartPaste trích xuất báo giá thông minh đạt chuẩn Mục 10.
5. Đáp ứng trọn vẹn 17 mã kiểm tra trong Phiếu chấm chi tiết của học phần (K1.1 → K5.3) với điểm số tự đánh giá đạt **100/100 điểm**.

9.2. Bài học kinh nghiệm

- **Hiểu sâu về Clean Architecture**: Nắm vững sự khác biệt giữa kiến trúc phân tầng truyền thống (Layered Architecture) và Clean Architecture/Onion Architecture; thấy rõ sức mạnh của nguyên lý đảo ngược phụ thuộc (DIP).
- **Kỹ năng thiết kế CSDL thực chiến**: Thiết kế bảng chuẩn 3NF, xử lý mối quan hệ khóa ngoại hai chiều phức tạp giữa POS và Kho hàng, viết Stored Procedure và View tối ưu hiệu năng.
- **Tư duy phòng thủ lỗi (Defensive Programming)**: Xử lý lỗi mềm, không để lộ exception ra ngoài màn hình người dùng, kiểm tra điều kiện biên chặt chẽ trước khi tác động vào CSDL.

9.3. Hướng phát triển tương lai

1. **Mở rộng mô hình Đa kho hàng (Multi-Warehouse)**: Quản lý luân chuyển hàng hóa giữa kho tổng và nhiều chi nhánh cửa hàng bán lẻ khác nhau.
2. **Tích hợp phần cứng Barcode Scanner**: Quét mã vạch trực tiếp bằng Camera điện thoại hoặc súng bắn mã vạch chuyên dụng thông qua thư viện JavaScript Barcode API trên Blazor.
3. **Đồng bộ sàn Thương Mại Điện Tử**: Xây dựng Webhook kết nối đồng bộ tồn kho tự động với các sàn TMĐT phổ biến (Shopee, TikTok Shop, Lazada).
4. **Phân tích dự báo tồn kho thông minh (Inventory Forecasting)**: Ứng dụng mô hình Machine Learning phân tích lịch sử bán lẻ POS để tự động gợi ý số lượng hàng cần nhập trước mỗi dịp lễ, Tết.

Báo cáo được hoàn thiện và xác nhận lưu trữ tại tệp `baocaodemo.md` *trong thư mục gốc của dự án WebKhoBanLe.*